

Projektuppgift

Frontend-baserad webbutveckling

Projektuppgift

Frida Ljungberg



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Frida Ljungberg, frlj2501@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: VT, 2026

Sammanfattning

Syftet med denna projektuppgift är att skapa en webbapplikation som visar en sökt plats på en karta samt vädret på platsen. En av problemställningarna för projektet var att lyckas med en så kallad "mashup", där data från två olika API:er kombineras för att visa önskat resultat. Med hjälp av AJAX-anrop hämtas information från de två olika API:erna i bakgrunden för att uppdatera sidan. Projektets andra problemställning är att få en strukturerad kod, då av tidigare erfarenheter kan det bli långa filer där det är svårt att leta reda på den kodsnuett man min ändra på. Med hjälp av tekniker inom SCSS, som exempelvis partials och nesting, blev det enklare att dela upp och strukturera koden. Båda problemställningarna blev lösta, dock kan det diskuteras om man tycker att koden är tillräckligt strukturerad eller ej.

Innehållsförteckning

Sammanfattning	iii
1 Introduktion	1
2 Teori	2
2.1 NPM	2
2.2 Vite	2
2.3 Sass och SCSS	2
2.4 Partials	2
2.5 Nesting	2
2.6 Mashup	2
2.7 JSDoc	3
3 Metod	4
4 Konstruktion	5
5 Resultat	8
6 Slutsatser	9
Källförteckning	10
Bilaga A: Designskiss	11

1 Introduktion

I detta projekt ska en webbapplikation skapas där man ska kunna söka på en plats och se platsens lokalisering på en karta samt dess nuvarande väder. Syftet med denna applikation är att man enkelt ska kunna se vädret på en särskild plats.

Ett problem som ska försökas lösas under projektets gång är att få till en "mashup", där information från olika källor ska kombineras och integreras med varandra. För att matcha applikationens syfte behövs då en källa som hämtar information om var platsen ligger och en annan källa som hämtar väderdata.

Ett annat problem som ska försökas lösas i detta projekt är att ha strukturerad kod. I tidigare projekt har man suttit med filer (särskilt med styling) som är långa och svåra att navigera i. Under denna kursens gång har vi fått träna på hur man kan strukturera upp sin kod, vilket är något som jag vill få till i detta projekt.

2 Teori

I detta kapitel kommer olika begrepp och tekniker att redogöras.

2.1 NPM

NPM är en förkortning för *Node Package Manager*. Denna förkortning används för att hantera JavaScript-paket och bibliotek. [1]

2.2 Vite

Vite är ett byggverktyg som används inom webbutveckling för att smidigt kunna utveckla frontend-applikationer. Syftet med att använda Vite är exempelvis att man enklare ska kunna utveckla moderna webbapplikationer och kunna bygga och optimera kod för produktion. [2]

2.3 Sass och SCSS

Sass är en CSS-preprocessor och är en av de mer populära. Den nya syntaxen inom Sass är SCSS, som är en förkortning för *Sassy CSS*. SCSS har samma syntax som CSS, vilket gör att det är enkelt att använda om man är van att koda CSS-kod. [3]

2.4 Partial

Med hjälp av partials i Sass kan man strukturera upp sin CSS-kod genom att dela upp sin kod i olika filer. På så sätt blir det enklare att hålla ordning på CSS-koden eftersom allt inte ligger i samma fil. Man skapar en partial genom att lägga ett understreck i början på SCSS-filnamnet. Man importerar sedan sina olika partials i en "main.scss-fil", vilket är den fil som kompileras så att all kod slås ihop till en enda CSS-fil. På så sätt minskar antalet HTTP-förfrågningar och förbättrar prestandan på webbplatser. [4]

2.5 Nesting

Med hjälp av nesting kan vi skriva styling för olika selektorer i en annan selektor. På så sätt slipper man att återupprepa samma selektor flera gånger. Dock ska man vara försiktig med att nestla för djupt, vilket kan resultera till att CSS-filen blir onödigt stor. [5]

2.6 Animering med hjälp av CSS

I CSS kan man skapa en animering som gör att man kan skapa visuella rörelser och övergång på en webbsida, utan att använda sig av JavaScript. En positiv aspekt med att använda CSS-animeringar är att de är prestandaeffektiva, då webbläsaren själv optimerar hur animationerna visas på skärmen. Animeringar används ofta för att förbättra upplevelsen för användarna, som exempelvis att ge feedback vid en interaktion. [6]

2.7 Mashup

En mashup inom webbutveckling är en webbsida eller webbapplikation som använder innehåll från mer än en källa för att skapa en ny tjänst. Mashup

antyder enkel och snabb integration och används ofta med hjälp av öppna API:er som kombineras. [7]

2.8 JSDoc

Som programmerare kan man använda sig av JSDoc för att skapa tydlig, strukturerad dokumentation direkt i källkoden, då JSDoc är ett dokumentationssystem. I JSDoc kan man exempelvis beskriva funktioners och variablers syfte. Genom att dokumentera sin JavaScript-kod med JSDoc kan andra utvecklare enklare förstå koden. [8]

3 Metod

I detta kapitel kommer de metoder som har används under projektets gång för att lösa problemställningarna att redovisas.

I förarbetet gjordes en enklare prototyp i *Figma*. När prototypen skapades bestämdes färgvalen som skulle användas på sidan. Inspiration för vilka färger som skulle användas på sidan hittades på webbplatsen *Colorhunt.co*.

Själva kodandet av webbplatsen gjorde i *Visual Studio Code*. För att skapa applikationens design användes SCSS, där olika partials användes för att strukturera upp koden. För att få ännu mer struktur och tydlighet i CSS-koden användes nesting.

För att uppdatera innehållet utan att behöva ladda om hela sidan har AJAX-anrop använts. För att dokumentera JavaScript-koden användes JSDoc.

4 Konstruktion

I detta kapitel kommer det redovisas hur det praktiska arbetet har sett ut under projektets gång.

4.1 Förarbete och start av utveckling av applikationen

Det första som gjordes var en enkel desginskiss för desktop i programmet *Figma* (se bilaga A). Det beslutades att ha en enkel design. Inspiration till färgvalen på sidan är hämtade från webbplatsen *colorhunt.co*. Valet hamnade i ett blått tema med en nyans av beige som bakgrundsfärg.

När designskissen var färdig var det dags att börja utveckla applikationen. För att arbetsprocessen skulle vara automatiserad installerades *Vite* genom kommandot `create vite@latest` i terminalen. När installationen var färdig togs de filer bort som "hängde med" vid installationen. Även SCSS installerades. Något som också installerades var *Vite-Plugin-Image-Optimizer*.

När allt var installerat var nästa steg att sätta upp en grundstruktur. Fyra olika partials skapades med namnen `_base.scss`, `_color.scss`, `_layout.scss` och `_animation.scss`. I filen `main.scss` som också skapades importerades de fyra partialsen. I `base` lades grunder i designen in, som style för elementen header, main och footer. Med hjälp av nesting strukturerades SCSS-koden. En *mixin* skapades med namnet *flex-center*, som inkluderades i headern, main och fottern. Syftet med *flex-center* var att få innehållet centrerat på webbsidan.

I partialen `colors` lades variabler in för de färger som skulle användas i applikationen. Det lades in variabler både för ljust och mörkt tema. Färgkoderna för ljust tema är hämtade från designskissen. För att bestämma vilka färgkoder som det mörka temat skulle användas "färgpaletten" med där färgen från början var samma färg som det lusa temat, men där jag tog musen och drog ner så det skulle bli mörkare. Med hjälp av *mediaquery* lades färgkod in om användaren har mörkt tema inställt på sin enhet. Lite senare i projektet lades även färger in för psedoklasserna *hover* och *active* på sökknappen, för att användaren skulle få feedback på sin interaktion.

I partialen `layout` lades kod in för responsiv design, även här med hjälp av en *mediaquery*.

4.2 JavaScript-kod

Nästa steg i projektet var att börja utveckla JavaScriptkod för att få till en mashup. Två olika API:er behövdes, ett för att få fram koordinater och ett för att läsa in väderdata. Båda API:erna som används i applikationen är från Open-Meteo. API:erna anropades via fetch-funktioner.

Funktionen `buildMap` tar emot koordinater och skapar en URL för den sökta platsen på en karta. Funktionen returnerar [URL:en](#) för den sökta platsen.

När användaren skriver in en plats som den vill veta vad det är för väder skickar funktionen *loadData* en förfrågan till API:et och väntar på svar. Eftersom koden är asynkron sker dessa förfrågningar i bakgrunden. Om platsen hittas visas den på kartan med hjälp av koordinaterna och funktionen *buildMap* bygger kartan.

Funktionen *getWeather* hämtar väderkod och temperatur från Open-Meteos *Weather Forecast API* [9]. För att översätta väderkoderna som hämtades från Open-Meteo skapades funktionen *interpretWeatherCode*, där varje väderkod översätts till en textsträng. Vad koderna stod för kunde man läsa om dokumentationen [9].

När appen provkördes under utvecklingens gång märkte jag att om man skrev en plats som inte existerar och klickade på sökknappen hände det ingenting. Det beslutades då att ett felmeddelande skulle visas. Med hjälp av en if-sats visas ett felmeddelande om ingen plats matchar användarens sökning, eller om man klickar på sökknappen när input-fältet är tomt. På så sätt får användaren feedback på sin interaktion, även om platsen inte finns.

Innan man klickade på sökknappen var kartan inte synlig, men kanten på iframe-element syntes på skärmen. Detta såg inte särskilt estetiskt ut enligt mig. För att få bort kanten sattes klassen *hidden* på iframe-elementet i filen *index.html*. I partialen *base* lades kod in för att dölja elementet. Sedan lades en kodsnuitt in i JavaScript-filen för att ta bort klassen *hidden* när funktionen *loadData* körs.

För att funktionerna ska köras lades en händelsehanterare in som triggas när man klickar på sökknappen.

För att dokumentera JavaScript-koden har JSDoc använts. I dokumentationen har exempelvis funktioner beskrivits. När koden var kommenterad kördes kommandot *npm jsdoc -r src -d docs* i terminalen för en automatisk dokumentering. Dokumentationen lades som en länk i applikationens footer.

4.3 Animering

För att få applikationen lite mer levande lades ett moln in som färdas över skärmen horisontellt. All animationskod lades i filen *animation.scss*. Molnet är skapat i *paint* och sedan frilagd i *Photopea*. Med en *keyframe* med namnet *moveCloud* gjordes animationen att molnet rörde sig från vänster till höger med hjälp av CSS-transformering. För att få molnet att kännas mer i bakgrunden gjordes den 25% synlig.

En animation lades också på den text där vädret och temperaturen står. När muspekaren är över texten blir den större och ändrar färg till en mörkröd nyans. Med hjälp av *ease-out* startar själva animationen snabbt och saktar ner mot slutet.

4.4 Publicering

Arbetet med publicering blev krångligare än tänkt. Den publiceringstjänst som har använts mest under utbildningens gång är *Netlify*. Dock, varnade tjänsten mig om att jag hade använt upp nästan all min måndaskredit. Med grund i detta testades publiceringstjänsten *Render*. När applikationen var publicerad genom *Render* kunde inte webbläsaren läsa in sidans style. Applikationen provades också att publicera genom *GitHub-pages*, men samma problem uppstod där. Efter forskning om varför inte styling-filen inte kunde läsas in verkade det som att man behöver bygga om applikationen till statiska filer [10]. Detta problem har inte stötts på vid publicering genom *Netlify*. Till slut testades applikationen att publiceras genom *Vercel*, och där fungerade stylingen utan problem. Med grund i detta valdes därför *Vercel* som publiceringstjänst.

5 Resultat

Ett av problemen som skulle lösas i denna projektuppgift var att få till en så kallad "mashup". För att få till applikationens syfte, att kunna läsa in väder och visa platsen på en karta, behövdes två olika API:er som samarbeta för att lyckas med detta. De två API:erna som användes i detta projekt är från *Open-Meteo*, en som hämtar koordinater för platser och en som hämtar väderdata. Eftersom resultatet av dessa två presenteras gemensamt i applikationen (platsen position på en karta samt platsens väder), har en mashup gjorts av dessa källor. Med grund i detta är projektets första problem löst.

Projektets andra problem som skulle försöka lösas var att ha strukturerad kod. Om detta är löst eller ej kommer vara av en subjektiv åsikt, då man kan tycka olika om en kod är bra strukturerad eller inte. Om man dock ska jämföra enkelheten att underhålla koden i detta projekt med föregående, är koden mer uppdelad än tidigare. Med grund i detta skulle jag säga att problemet är löst. Men, eftersom detta är en subjektiv åsikt kan andra dock tycka att problemet är inte är löst.

6 Slutsatser

Att använda sig av olika API:er i en applikation genom en mashup kan vara användbart när man utvecklar. De olika API:erna kan komplettera varandra, så som i det här projektet när en specifik plats på en karta samspelar med platsen väder. Genom att förfrågningar till dessa API:er sker asynkront blir det en positiv användarupplevelse. Något som jag upptäckte under utvecklingens gång var att det är viktigt med feedback när man använder applikationen, så som om man söker på en plats som inte finns att man får reda på det genom exempelvis ett felmeddelande. När inget felmeddelande dök upp satt man och väntade på att kartan skulle komma upp på skärmen, och då fick man vänta helt förgäves. När man i stället fick ett felmeddelande när man klickade på sök och platsen ej kunde hittas, kunde man direkt förstå varför kartan inte dök upp och testa att söka igen. Det här med meddelanden som feedback, eller annan feedback, på den interaktionen som man gör i en applikation är något jag kommer ta med mig i framtida projekt.

För att få till en bra struktur i sin kod är viktigt, både för en själv för att kunna underhålla koden, och om andra utvecklare ska underhålla ens skapta kod. I föregående kurser har jag märkt att i slutskedet av projekt är det svårt att hitta den kod man vill ändra då ens fil innehåller många rader. Med hjälp av tekniker vi har lärt oss under denna kurs, så som att använda partials för att strukturera SCSS-kod har hjälpt mycket. Att dela upp sin kod i olika partials gör att det blir enklare att hitta den kod man vill underhålla, i stället för att skriva allt i samma fil. Koden kändes också mer strukturerad vid användandet av nesting, eftersom all styling som skulle vara på exempelvis main och elementen inuti låg under main i SCSS-filen. Detta sätt att strukturera kod på är något som jag kommer att fortsätta arbeta med i kurser framöver.

Källförteckning

- [1] Mittuniversitetet. Moment 1 – Automatisering & publicering, NodeJs och NPM. Kurswebbplats Canvas [Internet]. Tillgänglig: https://canvas.miun.se/courses/6137/pages/raster-och-vektor-grafik?module_item_id=166491. Hämtad 2026-03-12.
- [2] Mittuniversitetet. Moment 1 – Automatisering & publicering, Automatisering med Vite. Kurswebbplats Canvas [Internet]. Tillgänglig: https://canvas.miun.se/courses/8221/pages/automatisering-med-vite?module_item_id=195261. Hämtad 2026-03-12.
- [3] Mittuniversitetet. Moment 3 – CSS-preprocessorer, Sass och SCSS. Kurswebbplats Canvas [Internet]. Tillgänglig: https://canvas.miun.se/courses/8221/pages/sass-och-scss?module_item_id=200012. Hämtad 2026-03-13.
- [4] Mittuniversitetet. Moment 3 – CSS-preprocessorer, Partials och filstruktur. Kurswebbplats Canvas [Internet]. Tillgänglig: https://canvas.miun.se/courses/8221/pages/partials-och-filstruktur?module_item_id=200018. Hämtad 2026-03-13.
- [5] Mittuniversitetet. Moment 3 – CSS-preprocessorer, Nesting. Kurswebbplats Canvas [Internet]. Tillgänglig: https://canvas.miun.se/courses/8221/pages/nesting?module_item_id=200014. Hämtad 2026-03-13.
- [6] Mittuniversitetet. Moment 4 – Grafiska effekter med CSS, Vad är en animering?. Kurswebbplats Canvas [Internet]. Tillgänglig: https://canvas.miun.se/courses/8221/pages/vad-ar-en-animering?module_item_id=200021. Hämtad 2026-03-13.
- [7] Wikipedia. Mashup [Internet]. Utgivningsort: Utgivare; År (2026-01-20) [2026-03-13]. Hämtad från: [h https://en.wikipedia.org/wiki/Mashup_\(web_application_hybrid\)](https://en.wikipedia.org/wiki/Mashup_(web_application_hybrid)).
- [8] Mittuniversitetet. Moment 5 – Dynamiska webbplatser, JSDoc. Kurswebbplats Canvas [Internet]. Tillgänglig: https://canvas.miun.se/courses/8221/pages/jsdoc?module_item_id=199578. Hämtad 2026-03-13.
- [9] Weather Forecast API [Internet]. År (2026) [2026-03-12]. Hämtad från: https://open-meteo.com/en/docs?hourly=¤t=weather_code.
- [10] Vite. Deploying a Static Site [Internet]. [2026-03-21]. Hämtad från: <https://vite.dev/guide/static-deploy>.

Bilaga A: Designskiss

